

MTH401E - Homework 1

Sena ERSOY
040200434

March 15, 2026

1 Understanding the Instruction Set Architecture

The initial phase of this assignment required a careful analysis of the provided instruction table. I realized this assignment uses a custom Instruction Set Architecture (ISA) specifically designed for this homework.

Unlike standard RISC-V where the `opcode` is located at the lowest bits [6:0], this custom ISA places the opcode in the middle of the instruction at bits [19:13]. Furthermore, the `funct3` field was moved to bits [7:5], the register indices (`rd`, `rs1`, `rs2`) were shifted, and the immediate assembly structures were heavily modified. The specific mappings for operation types (like branch condition codes and load variations) were also uniquely scrambled compared to standard RISC-V. Recognizing this custom encoding was the critical first step to implementing the correct bitwise extractions.

2 Decoder Design Workflow

The goal was to design a fully combinational decoder in SystemVerilog (`decoder.sv`) that parses a 32-bit instruction and outputs a populated `dinstr_t` struct. To ensure the synthesis tool would not infer unwanted latches, the workflow followed a strict combinational logic pattern:

1. Field Extraction: The common bit fields (`opcode`, `funct3`, `funct7`) and register indices were extracted at the top of the module using continuous assignments.
2. Default Assignments: At the beginning of the `always_comb` block, every single signal in the output struct (e.g., `valid`, `is_mem`, `is_pc_changer`, `imm`) was assigned a default value (mostly 0). This guarantees that every output has a defined state in every logic path.
3. Instruction Decoding: A `case` statement evaluated the extracted 7-bit `opcode`. Depending on the opcode (R-Type, I-Type, S-Type, B-Type, U/J-Type), the relevant flags were overwritten to 1.
4. Nested Logic & Immediates: Inside each opcode case, a nested `case` statement evaluated `funct3` (and sometimes `funct7`) to determine the exact operation (e.g., mapping to the `ADD` or `BEQ` enums). Finally, the custom immediate values were assembled using bit concatenation and sign-extension (or zero-extension for shift operations).

3 Vivado Implementation and Syntax Challenges

To verify the hardware logic, I set up an RTL project in Xilinx Vivado 2018.3, adding the source files and the package definition. During compilation and elaboration, I encountered several strict syntax enforcement issues unique to the Vivado simulator (XSim):

- Semicolon Error: Vivado's synthesizer flagged a syntax error due to an extra semicolon after the `endpackage` keyword in `typedef_pkg.sv`, which had to be removed.
- Abstract Enum Typing: The package defined the operations as `typedef enum [5:0] { ... }`. However, Vivado 2018.3 does not fully support packing abstract enums into structs for simulation. It threw an IEEE 1800 syntax error because the underlying data type was missing. To allow the simulation to elaborate, I had to explicitly cast the enum to a logic vector in `typedef.pkg`: `typedef enum logic [5:0] { ... }`.

4 RTL Schematic Analysis

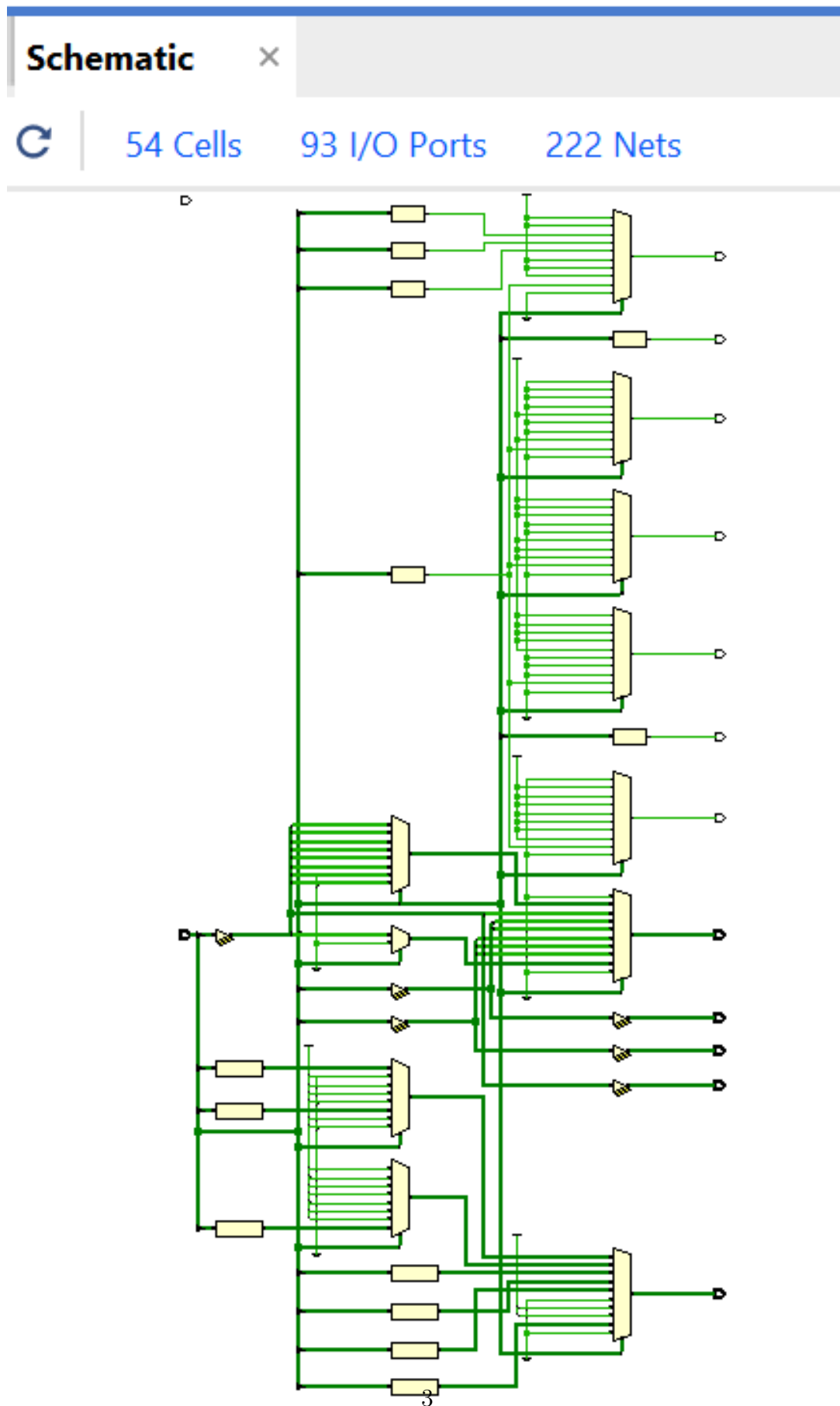


Figure 1: RTL Schematic of the Custom Instruction Decoder

Looking at the elaborated RTL schematic (Figure 1), the purely combinational nature of the design becomes highly visible. Because there are no clock signals or flip-flops used inside the module, the schematic consists entirely of routing logic. We can see large multiplexers routing the correct concatenated immediate bits to the output based on the opcode. Furthermore, equality comparators evaluate the `opcode`, `funct3`, and `funct7` wires, feeding into combinatorial logic gates that drive the single-bit output flags like `is_mem` and `is_pc_changer`.

5 Testbench Algorithm (`tb_decoder.sv`)

The provided testbench automates the validation of the decoder by generating 43 distinct instructions and comparing the decoder’s output against an array of expected structs (`expected_dinstructions`).

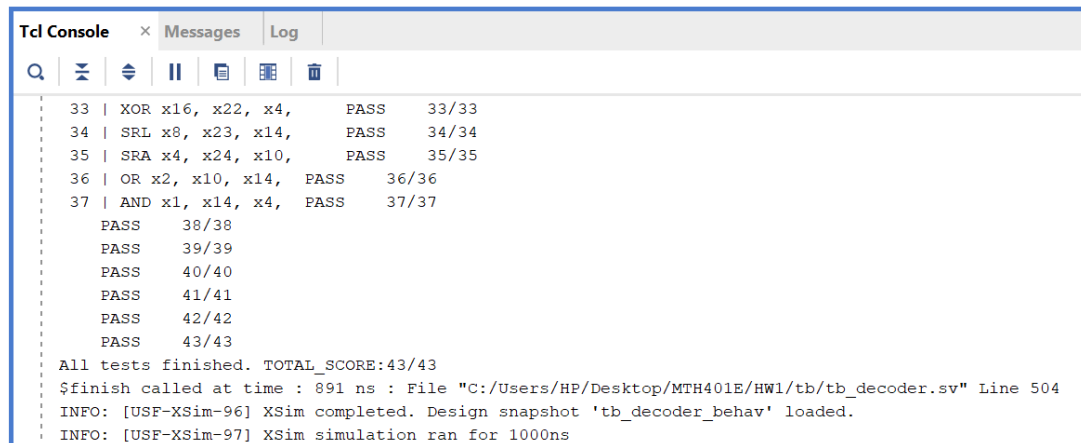
Instead of hardcoding 32-bit hex values, the testbench dynamically builds each instruction bit-by-bit inside a `for` loop. It uses arrays of logic vectors (e.g., `opcode_set`, `rd_or_imm5`, `func3t`) to populate the specific bit ranges of an empty 32-bit variable. It relies on internal "valid" bits stored within those arrays to determine if a specific field (like an I-Type immediate or a `funct3` code) should be injected into the instruction for that specific test case. Once the instruction is built, it is fed into the decoder, and a custom `compare_dinstr` function evaluates the results.

6 Testbench Solutions and Logic Implementation

Getting the testbench to simulate correctly required precise mapping of the instruction fields and strict handling of the array indices.

- Placeholder Replacement: The original file contained undeclared `xxxx` placeholders in the instruction-building loop. I replaced these with the exact bit slices defined by the custom ISA (e.g., `inst[19:13]` for the opcode).
- Validity Bit Indexing: The testbench formatted the `funct3` data array using an underscore, such as `4'b100_1`. In SystemVerilog, the underscore is ignored by the compiler and serves only as a visual aid for human readability. Therefore, the value `4'b100_1` is treated exactly as the 4-bit binary number 1001. According to the schema defined in the testbench comments, the least significant bit (`[0]`) represents the valid flag, while the three most significant bits (`[3:1]`) hold the actual `funct3` data. To correctly inject this data into the instruction, the testbench logic was written to check `func3t[i][0]` for validity and to extract `func3t[i][3:1]` as the data to insert into the instruction. (When I first tried parsing, I misunderstood the valid flag, thought it was `func3t[i][3]` and many of my tests failed, then I used AI to find my error here. And it showed me this mistake of mine. Thus I admit I used AI for debugging)

7 Simulation Output and Analysis

The image shows a screenshot of the Vivado Simulation Console. The window has tabs for 'Tcl Console', 'Messages', and 'Log'. The 'Tcl Console' tab is active, displaying a list of test results. Each line shows a test number, an instruction, registers, a status, and a score. Tests 33 through 43 are listed, all with a status of 'PASS'. The final line indicates 'All tests finished. TOTAL_SCORE:43/43'. Below this, there are informational messages about the simulation time and design snapshot.

```
Tcl Console x Messages Log
Q [ ] [ ] [ ] [ ] [ ] [ ] [ ]
33 | XOR x16, x22, x4, PASS 33/33
34 | SRL x8, x23, x14, PASS 34/34
35 | SRA x4, x24, x10, PASS 35/35
36 | OR x2, x10, x14, PASS 36/36
37 | AND x1, x14, x4, PASS 37/37
PASS 38/38
PASS 39/39
PASS 40/40
PASS 41/41
PASS 42/42
PASS 43/43
All tests finished. TOTAL_SCORE:43/43
$finish called at time : 891 ns : File "C:/Users/HP/Desktop/MTH401E/HW1/tb/tb_decoder.sv" Line 504
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_decoder_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

Figure 2: Vivado Simulation Console Output demonstrating 43/43 Passed Tests

The simulation flow executes in three steps: compilation (syntax checking), elaboration (building the virtual circuit), and simulation (running time). In the console output (Figure 2), we can observe the testbench pushing instructions sequentially. For every valid instruction, it prints the matched operation name, the parsed registers, and the reconstructed immediate value, followed by a **PASS**.

Notably, tests 38 through 43 show a "silent" pass. Because my decoder successfully caught the corrupted `funct7` bits and flagged the instructions as invalid (`valid = 0`), the testbench bypassed its print statement but correctly evaluated the behavior as a **PASS**, resulting in a perfect **TOTAL_SCORE: 43/43**.

8 Conclusion

This assignment provided deep hands-on experience with hardware description languages and custom computer architecture. By deviating from the standard RISC-V ISA, it forced a deeper understanding of bitwise instruction parsing, multiplexing, and combinational logic design. Furthermore, developing the design in Vivado highlighted the differences between writing generic code and writing code that complies strictly with hardware synthesis and simulation standards.